Min-Heap Algorithms Reference Manual

0-INDEXED ARRAY IMPLEMENTATION & VERIFIED PSEUDOCODE

1. Helper & Indexing Functions (0-Based Array Indexing)

With 0-based indexing (root at index 0), navigation formulas for node index `i` are calculated as follows:

```
PARENT(i) = ⌊(i - 1) / 2⌋     LEFT(i) = 2i + 1              RIGHT(i) = 2i + 2
```

2. Bottom-Up Min-Heap Construction — O(n) Time

Builds a min-heap in linear time by calling `MIN-HEAPIFY` backward from index `⌊n / 2⌋ - 1` down to 0.

MIN-HEAPIFY(A, i) — `O(log n)`

```
1  l = LEFT(i)
2  r = RIGHT(i)
3  if l < A.heap-size and A[l] < A[i]
4      smallest = l
5  else smallest = i
6  if r < A.heap-size and A[r] < A[smallest]
7      smallest = r
8  if smallest != i
9      exchange A[i] with A[smallest]
10     MIN-HEAPIFY(A, smallest)
```

BUILD-MIN-HEAP(A) — `O(n)`

```
1  A.heap-size = A.length
2  for i = ⌊A.length / 2⌋ - 1 downto 0
3      MIN-HEAPIFY(A, i)
```

3. Top-Down Min-Heap Construction — O(n log n) Time

Builds the heap incrementally by expanding valid boundaries and bubbling elements up via `HEAP-DECREASE-KEY`.

HEAP-DECREASE-KEY(A, i, key) — `O(log n)`

```
1  if key > A[i]
2      error "new key is larger than current
key"
3  A[i] = key
4  while i > 0 and A[PARENT(i)] > A[i]
5      exchange A[i] with A[PARENT(i)]
6      i = PARENT(i)
```

MIN-HEAP-INSERT(A, key) — `O(log n)`

```
1  A.heap-size = A.heap-size + 1
2  A[A.heap-size - 1] = ∞
3  HEAP-DECREASE-KEY(A, A.heap-size - 1, key)
```

3. Top-Down Min-Heap Construction — O(n log n) Time

Builds the heap incrementally by expanding valid boundaries and bubbling elements up via `HEAP-DECREASE-KEY`.

HEAP-DECREASE-KEY(A, i, key) — `O(log n)`

```
1  if key > A[i]
2      error "new key is larger than current
key"
3  A[i] = key
4  while i > 0 and A[PARENT(i)] > A[i]
5      exchange A[i] with A[PARENT(i)]
6      i = PARENT(i)
```

MIN-HEAP-INSERT(A, key) — `O(log n)`

```
1  A.heap-size = A.heap-size + 1
2  A[A.heap-size - 1] = ∞
3  HEAP-DECREASE-KEY(A, A.heap-size - 1, key)
```

BUILD-MIN-HEAP-TOP-DOWN(A) (In-Place)

`O(n log n)`

```
1  A.heap-size = 1
2  for i = 1 to A.length - 1
3      HEAP-DECREASE-KEY(A, i, A[i])
```

BUILD-MIN-HEAP-ONLINE(A) (Dynamic Stream)

`O(n log n)`

```
1  n = A.length
2  A.heap-size = 0
3  for i = 0 to n - 1
4      MIN-HEAP-INSERT(A, A[i])
```

4. Min-Priority Queue Operations

HEAP-MINIMUM(A) `O(1)`

```
1  return A[0]
```

HEAP-EXTRACT-MIN(A) `O(log n)`

```
1  if A.heap-size < 1
2      error "heap underflow"
3  min = A[0]
4  A[0] = A[A.heap-size - 1]
5  A.heap-size = A.heap-size - 1
6  MIN-HEAPIFY(A, 0)
7  return min
```

5. Comparative Analysis Summary

Construction Strategy	Algorithm	Index Range	Time Complexity	Sifting Operation
Bottom-Up	BUILD-MIN-HEAP	i = ⌊n/2⌋ - 1 downto 0	`O(n)`	Sifts down (MIN-HEAPIFY)
Top-Down In-Place	BUILD-MIN-HEAP-TOP-DOWN	i = 1 to n - 1	`O(n log n)`	Sifts up (HEAP-DECREASE-KEY)
Online Insertion	BUILD-MIN-HEAP-ONLINE	i = 0 to n - 1	`O(n log n)`	Sifts up (MIN-HEAP-INSERT)